



PROGRAMACIÓN iii

Resolución de conflictos

- A lo largo del desarrollo colaborativo, hemos comprobado que Git posee algoritmos sumamente eficientes para unificar el trabajo de múltiples ramas de forma automática.
- Sin embargo, existe un escenario crítico en la concurrencia de código: ¿qué sucede si dos programadores modifican exactamente la misma línea del mismo archivo al mismo tiempo? Al enfrentarse a esta superposición, Git —que es un motor lógico y carece de criterio humano— no puede tomar la decisión arbitraria de priorizar un bloque de código sobre otro.

Resolución de conflictos

- Cuando esto ocurre, el sistema detiene inmediatamente el proceso de fusión y declara un Conflicto de Integración (Merge Conflict).
- Lejos de representar un fallo o un error del sistema, este evento es un mecanismo de seguridad fundamental de la ingeniería de software: la herramienta nos advierte que existen dos verdades excluyentes y delega la responsabilidad arquitectónica al desarrollador, exigiendo su intervención manual para determinar la versión definitiva.

Resolución de conflictos



- Para desmitificar este escenario y aprender a gestionarlo con profesionalismo, simularemos un conflicto de manera controlada.
- 1. El ejercicio comienza asegurándonos de estar anclados en nuestra rama principal mediante:
git checkout main.
- 2. A continuación, abriremos el archivo *texto_1.txt* (creado en la práctica de aislamiento) y sobrescribiremos todo su contenido con una única línea que dicte: *"Este es el texto definitivo de la rama main"*.



Resolución de conflictos



3. Consolidamos esta modificación en el historial ejecutando:

```
git add texto_1.txt
```

```
git commit -m "Se edita texto_1 desde main"
```

4. Inmediatamente después, viajamos a nuestra línea de tiempo secundaria ingresando:

```
git checkout primera_rama
```



Resolución de conflictos



5. Al abrir exactamente el mismo archivo *texto_1.txt*, borraremos su contenido y escribiremos una afirmación contraria en la misma línea: *"Este es el texto exclusivo de la primera rama"*.
6. Registramos este cambio divergente utilizando:

```
git add texto_1.txt
```

```
git commit -m "Se edita texto_1 desde primera_rama"
```

Con estas acciones, el escenario conflictivo está preparado, ya que poseemos dos historiales técnicos que afirman cosas distintas sobre la misma porción de un archivo.



Resolución de conflictos



7. Para forzar la colisión, debemos retornar a la rama receptora ejecutando:

git checkout main

8. E invocar la orden de unificación

git merge primera_rama

Al detectar la superposición, la terminal interrumpirá la operación y emitirá una alerta crítica indicando que la fusión automática ha fallado (Automatic merge failed; fix conflicts and then commit the result).



Resolución de conflictos



9. Como herramienta de diagnóstico adicional, si en este instante ejecutamos:

git status

Observaremos que el archivo *texto_1.txt* se resalta en color rojo bajo la advertencia both modified (modificado por ambos).

- Esta pausa es la conducta esperada. Para permitirnos tomar una decisión, Git ha modificado temporalmente el código fuente del archivo inyectando marcadores visuales.



Resolución de conflictos



- Si abrimos *texto_1.txt* en nuestro editor, encontraremos símbolos estructurales específicos: <<<<<< *HEAD*, que señala el punto de inicio del código correspondiente a la rama en la que estamos parados actualmente (*main*).
- Una línea divisoria =====, que separa nuestra versión de la versión entrante.
- Finalmente >>>>>> *primera_rama*, que marca el cierre del bloque de código que intentamos fusionar.



Resolución de conflictos



- La resolución de un conflicto es, en esencia, un ejercicio de edición de texto puro y análisis lógico.
- El deber del desarrollador es intervenir el archivo, eliminar manualmente todos los marcadores estructurales inyectados por Git (<<<<<<<, =====, >>>>>>>) y estructurar el código exactamente como desean que quede en la versión de producción.
- Se puede optar por conservar el código de main, el de la rama secundaria, o realizar una síntesis de ambos.



Resolución de conflictos



10. Para esta ejercitación, borraremos los marcadores y redactaremos una línea conciliadora: *"Este archivo ahora contiene los aportes de main y también de la primera_rama"*. Tras guardar el archivo con su forma definitiva, debemos notificarle al sistema que el conflicto lógico ha sido subsanado.



Resolución de conflictos



11. Retornando a la terminal, preparamos el archivo limpio con (eliminará la advertencia roja en el status):

git add texto_1.txt

12. Y cerramos el ciclo ejecutando:

git commit -m "Conflicto resuelto en texto_1"

Al encontrarse el repositorio en un estado de pausa por fusión, Git asociará automáticamente este commit a la resolución exitosa de la colisión.



Resolución de conflictos



- Comprender que los conflictos son eventos rutinarios —y que su solución requiere simplemente leer la consola, auditar el código con calma y consolidar la síntesis— es una competencia irremplazable en la formación de cualquier analista de sistemas.





**Instituto Superior
Del Milagro**

NIVEL SUPERIOR TERCARIO

Nº 8207